

ITEX 213 Principles of Internet Technologies and Web Applications

Course Title: Principles of Internet Technologies and Web Applications **Course Code:** ITEX 213

Credit Hours: 3 (2 Th + 1 PR)

Nature: Theory + Practical

1. Course Description and Goals

This course introduces students to the foundations of internet technologies and the essential tools used to build modern web applications. It begins with core concepts such as the evolution of the web, the client–server model, and the role of protocols like HTTP/HTTPS, TCP/IP, UDP and DNS in enabling communication across networks. Students learn to create structured and accessible web pages using HTML, apply styling and layout principles with CSS, and add interactivity through JavaScript, including basic DOM manipulation, events, and data handling. The course also covers XML and JSON as data exchange formats and introduces server-side scripting with PHP. Students work with form processing, input validation, and database connectivity using MySQL to implement simple CRUD operations. Security fundamentals such as SQL injection risks, safe input handling, and the distinction between authentication and authorization are emphasized throughout. Ethical considerations related to privacy, accessibility, and inclusivity are integrated into discussions and practice. By the end of the course, students will be able to design and develop secure, functional, and user-centered web applications while understanding the broader technological and societal context in which today’s internet operates.

2. Learning Outcomes

On completion of this course, the students will be able to:

- Explain how the Internet and the Web function, including DNS, core protocols, and the client–server model.
- Identify the evolution of the web and interpret how modern sites deliver dynamic content.
- Build well-structured webpages using HTML and organize multimedia, forms, and links effectively.
- Style and layout pages using CSS, apply responsive design, and work with common selectors, the box model, and modern layout techniques.
- Write JavaScript to manipulate the DOM, handle events, validate input, use built-in objects, and exchange data using JSON.
- Use XML for structured data and apply related tools such as DTD, XML Schema, XPath, and XSLT.
- Develop server-side scripts in PHP, process form data securely, connect to MySQL, and perform CRUD operations.
- Recognize basic security risks, including SQL injection, and apply safe coding habits.

3. Content with specific objectives

Specific objectives	Contents
----------------------------	-----------------

• Understand what the Internet and the Web are, how they differ, and how the web has evolved from Web 1.0 to more interactive and data-driven versions. • Identify the structure of a URL and explain how domains and DNS help a browser locate websites. • Describe the basic client–server model, compare 1-tier, 2-tier, and 3-tier architectures, and distinguish between static and dynamic web content. • Explain how core protocols such as TCP/IP, UDP, HTTP, and HTTPS enable communication between browsers and servers. • Develop a clear, early understanding of APIs, REST-style thinking, and why modern applications use JSON for data exchange.	Unit 1: Web Fundamentals (5 Hours) 1.1 The Internet and the Web - Internet vs. World Wide Web. - Brief history and evolution of the web. - Web pages, websites, and web applications. - Web 1.0, Web 2.0 and Web 3.0. 1.2 URLs, Domains, and DNS - Structure of a URL. - Domain names and DNS lookup. - Role of DNS in reaching websites. 1.3 Client–Server Architecture - Basic client–server model. 1-tier, 2-tier, and 3-tier architectures. - Static vs. dynamic content. - High-level view of modern web app architecture. 1.4 Core Internet and Web Protocols - TCP/IP and UDP. - HTTP request–response cycle. - Introduction to HTTPS and TLS/SSL. 1.5 APIs and JSON in Modern Web Applications - What an API is and why modern applications rely on them? - REST-style thinking at a high level. - JSON vs. HTML vs. XML: what each is used for. - Simple example of a JSON response from a web service.
• Understand the purpose of HTML in web development and recognize how an HTML document is structured. • Use essential HTML elements to organize and format text clearly and meaningfully. • Create hyperlinks, embed images, and distinguish between absolute and relative URLs.	Unit 2: Hyper Text Markup Language (7 Hours) 2.1 Introduction to HTML and Document Structure - Introduction to HTML: - Overview of HTML's role in web development and the client-side structure of web pages. - The function of HTML in creating structured web content. - Structure of an HTML document including meta tags, elements and attributes, writing Comments. 2.2 Basic HTML Elements and Text Formatting - Headings, Text Formatting: Paragraphs and Preformatted Text elements, Presentational Elements, Phrase Elements, Editing Elements, Grouping Elements.

• Build tables, lists, and embed external content such as maps or videos using iframes. • Construct forms using common form controls and apply basic HTML5 validation attributes. • Work with multimedia elements and apply semantic HTML5 sectioning tags to improve structure and accessibility.	2.3 Hyperlinks, Images, and URL Types - Hyperlinks: - The <a> element for linking. - Difference between absolute and relative URL with examples. - Images: - Embedding images with the tag. - Difference between ismap and usemap attributes with examples. 2.4 Tables, Lists, and Embedding External Content - Tables and Nested tables with attributes like colspan and rowspan. - Lists: - Unordered lists, ordered lists, and definition lists. - Iframes for Embedding External Content: - Embedding external content using <iframe>. - Embedding Google Maps, YouTube videos, and other web pages. 2.5 Forms and Form Elements - Forms Overview: - Using <form> to create user input forms. - Form attributes: action, method and enctype. - Form Elements: - Input Elements: <input> types, <textarea>, <select>, <option>, <button>, <label>. - Fieldset and Legend: Grouping form elements with <fieldset> and adding a caption with <legend>. - Validation: - HTML5 input validation: Required fields, pattern matching with pattern, and range control. 2.6 Multimedia and HTML5 Sectioning Elements - Working with Multimedia: - Embedding audio and video using <audio> and <video> tags. - Audio/video attributes: controls, autoplay, loop, and handling supported formats. - HTML5 Sectioning Elements: - Using semantic HTML5 elements to structure content: <article>, <section>, <nav>, <header>, <footer>, <aside>.
--	--

- Understand the purpose of CSS and how it separates presentation from structure in web design.
- Write basic CSS rules using selectors, properties, and values, and apply styles through inline, internal, and external methods.
- Use a range of selectors such as including attribute, combinator, pseudo-class, and pseudo-element selectors to target elements effectively.
- Apply color, background, and text-styling properties to enhance visual design.
- Explain the CSS box model and adjust spacing, sizing, and overflow to control layout.
- Work with modern layout techniques, including display properties, positioning, floats, flexbox, and grid.
- Implement responsive design using fluid layouts, breakpoints, and media queries to adapt pages across devices.

Unit 3: Cascading Style Sheets (7 Hours)

3.1 Introduction to CSS and Basic Syntax

- Introduction to CSS:
 - Definition of CSS and its importance in web design for separating content from presentation.
- CSS Syntax:
 - Basic structure of CSS rules: selectors, properties, and values.
- Inserting CSS:
 - Inline CSS, Internal CSS and External CSS with examples.
- Writing Comments in CSS.

3.2 Selectors in CSS

- Basic Selectors, Attribute and Combinator Selectors, Pseudo-class and Pseudo-element Selectors.

3.3 Colors, Backgrounds, and Text Styling

- Colors in CSS:
 - Applying colors using keywords, hexadecimal codes and RGB.
- Backgrounds:
 - Applying background colors and images using the background property.
 - Controlling background repeat, position, and size.
- Text Styling:
 - Properties for controlling text appearance.

3.4 The Box Model and Layout Techniques

- The CSS Box Model:
 - Understanding the box model and its components.
- Box Model Properties:
 - Adjusting box model properties using margin, padding, border, and box-sizing.
 - box-sizing: border-box vs. box-sizing: content-box.
- Height and Width:
 - Setting element dimensions using height, width, min-width, max-width, min-height, max-height.

	iv. Overflow: a. Managing content that overflows its container using the overflow property (visible, hidden, scroll, auto). v. Layout Concepts: a. Intro to modern layout concepts: flexbox and grid. 3.5 Positioning, Float, and Alignment i. Position Property: a. Static, Relative, Absolute, Fixed and Sticky Positioning. ii. Float and Clear: a. Using the float property to align elements to the left or right of their container. b. Using clear to control how floated elements interact with other elements. iii. Display: a. Controlling element layout with the display property (block, inline, inline-block, none). 3.6 Responsive Web Design and Media Queries i. Responsive Web Design Principles: a. Building websites that adapt to different screen sizes and devices. b. Using flexible grids, layouts, and images for responsiveness. ii. Media Queries: a. Syntax for media queries and how they enable responsive design (@media). b. Targeting specific screen sizes (e.g., max-width, min-width). c. Examples of common breakpoints for different devices (e.g., tablets, mobile, desktop). d. Practical implementation of responsive design using media queries.
--	---

- Understand what JavaScript does in the browser and how it adds interactivity to web pages.
- Use modern JavaScript syntax to declare variables, work with different data types, and apply common operators.
- Write clear control structures and functions to handle basic logic and reuse code effectively.
- Work with built-in JavaScript objects, including strings, dates, and arrays, to process and manipulate data.
- Select, update, and reorganize elements on a webpage using core DOM methods.
- Handle user interactions through event listeners and perform basic error handling and client-side validation using regex where appropriate.
- Understand how cookies store small pieces of data and how they are created, read, and removed in JavaScript.
- Use JSON to structure, store, and exchange data, including parsing JSON from simple mock APIs.
- Build a high-level understanding of why JavaScript libraries and frameworks were created and recognize the roles of jQuery, React,

Unit 4: Client-Side Scripting with JavaScript (15 Hours)

4.1 Introduction to JavaScript and Adding JavaScript to a Page

- Introduction to JavaScript:
 - Overview of JavaScript as a programming language for the web.
 - Role of JavaScript in adding interactivity to web pages.
 - Understanding JavaScript as a client-side language executed in the browser.
- Adding JavaScript to a Page:
 - Inline, Internal and External JavaScript
- Output in JavaScript:
 - `document.write()`, `console.log()` and `alert()`.

4.2 Variables, Data Types, and Operators

- Variables:
 - Declaring variables using `var`, `let`, and `const`.
 - Scoping rules for variables (block scope and function scope).
- Data Types:
 - Primitive and Non-primitive data types
 - The implications of type coercion and type conversion.
- Operators:
 - unary, binary and ternary operators.

4.3 Control Statements and Functions

- Control Statements:
 - Conditional Statements: `if`, `else`, `else if`, and `switch` for decision-making in code.
 - Loops: `for`, `while`, and `do...while` loops for repeated execution of code blocks.
- Functions:
 - Declaring functions using function declarations and expressions.
 - Function parameters and return values.
 - Named, anonymous and arrow functions.

4.4 Built-in Objects

- Date Object: Creating and manipulating dates, getting and setting date and time values.
- String Object: String manipulation methods.

and modern frameworks like Vue or Angular.	iii. Array Object: a. Creating and manipulating arrays in JavaScript (e.g., push, pop, shift, unshift). b. Iterating over arrays using loops and the forEach() method. c. Common array methods examples. 4.5 Document Object Model (DOM) Manipulation i. The Document Object Model (DOM): a. Introduction to the DOM and its hierarchical structure. b. Selecting elements using getElementById(), getElementsByClassName(), getElementsByTagName() querySelector() and querySelectorAll(). ii. Modifying the DOM: a. Changing element content using innerHTML, textContent and changing style. b. Modifying attributes: setAttribute() and getAttribute(). 4.6 Event and Error Handling, Form Validation, and Cookies i. Event Handling: a. Capturing user interactions with event handlers (e.g., onsubmit, onclick, ondblclick, onload, onmouseover, onmouseout, onmouseup, onmousedown). b. Using addEventListener() for more flexible event handling. ii. Error Handling (Try/Catch) iii. Form Validation: a. Client-Side Validation: Ensuring form fields are filled out correctly before submission. b. Using Regular Expressions (Regex): Introduction to regex for pattern matching. Validating input fields like text, emails, phone numbers, and passwords with regex patterns. iv. Handling Cookies: a. What are cookies, and why they are used in web development? b. Creating, reading, and deleting cookies using JavaScript.
---	---

	4.7 JSON and Data Handling i. Introduction to JSON: a. Overview of JSON and its role in web development. b. JSON Structure: Understanding Key-Value Pairs and Supported Data Types. ii. Working with JSON a. Storing single and multiple data in JSON. b. Accessing and updating JSON values (including nested objects and arrays). c. Fetching and handling JSON from a mock API. iii. Conversion Methods a. Converting JavaScript objects to JSON strings and JSON strings to JavaScript objects. 4.8 Introduction to JavaScript Libraries and Frameworks i. Why Libraries and Frameworks Emerged? ii. Overview of Popular Technologies: a. jQuery (its historical role in simplifying DOM manipulation and AJAX). b. React (component-based thinking, state handling, unidirectional data flow, and server-side rendering). c. Vue or Angular (examples of full-featured ecosystems for building structured, scalable applications).
• Understand why XML was created and how it is used to store, structure, and exchange data across different systems. • Recognize the key differences between XML and HTML in terms of purpose and design. • Learn the basic rules that make an XML document well-formed, including proper nesting, naming, and attribute usage.	Unit 5: Extensible Markup Language (4 Hours) 5.1 XML Basics i. Introduction to XML: a. Overview of XML as a markup language designed for data storage and transport. b. Key differences between XML and HTML. c. Applications of XML in web services, configuration files, and data exchange. ii. XML Syntax Rules: a. Well-formed XML document structure, including the XML declaration. b. Basic syntax rules iii. XML Elements: a. Definition and structure of XML elements.

• Identify when to use elements versus attributes, and understand the role of namespaces in avoiding naming conflicts. • Explore common real-world applications of XML, such as configuration files, data transfer formats, and legacy web services. • Gain a high-level understanding of validation tools like DTD and XML Schema, and how they help ensure consistent structure. • Become familiar with XPath as a way to navigate XML documents, XSLT as a transformation tool, and XQuery as a method for querying XML data.	b. Rules for naming elements and hierarchy in XML. c. Importance of elements in structuring data and organizing content. iv. XML Attributes and Namespaces: a. Purpose of attributes in providing additional information to elements. b. Syntax and rules for attributes (enclosed in quotes). c. Best practices: When to use attributes vs. nested elements for clearer structure. d. Concept of XML namespaces v. Use Cases of XML 5.2 Validation and XML Related Technologies i. DTD and XML Schema (XSD) a. Why validation is needed? b. High-level view of how schemas define allowed structure and data types ii. XPath a. How it helps you navigate XML like folder paths in a file system? iii. XSLT a. Transforming XML into HTML, text, or even other XML formats. iv. XQuery a. Querying XML data, the way SQL queries databases.
• Understand the role of server-side scripting and how PHP works alongside HTML, CSS, and JavaScript to generate dynamic content. • Learn to set up a PHP environment (XAMPP, WAMP, MAMP, LAMP, or cloud-based tools) and write simple scripts that run through a web server. • Handle form submissions using PHP superglobal variables and apply basic	Unit 6: Server-Side Scripting using PHP with Database Connectivity (7 Hours) 6.1 Server-Side Concepts and PHP Basics i. What server-side scripting does and how it works with HTML, CSS, and JavaScript? ii. PHP in common development environments: XAMPP, WAMP, MAMP, LAMP, or cloud-based setups. iii. Writing a basic PHP script using variables, output statements, and comments. iv. Understanding how PHP files are executed through a server rather than directly in the browser. 6.2 Handling HTTP Requests and Form Processing i. Retrieving user input with PHP superglobals: \$_GET and \$_POST.

validation and sanitization to keep user input reliable and safe. • Build a practical foundation for connecting PHP to a MySQL database and interacting with tables through real examples. • Perform essential CRUD operations using PHP and structured SQL queries. • Develop basic awareness of common security concerns in server-side code, including SQL injection, prepared statements and the difference between authentication and authorization.	ii. Building a simple form in HTML and sending the data to a PHP script. iii. Basic server-side validation: checking if fields are empty, trimming input, and verifying basic formats. iv. Performing basic input sanitization using built-in functions to keep user input clean and safe. 6.3 Performing CRUD Operations i. How PHP communicates with a database in a real-world web application? ii. Basics of relational tables and primary keys using MySQL for demonstration. iii. Establishing a connection using MySQLi or PDO (any one approach). iv. Implementing simple CRUD operations: a. Create: Insert form data into a database table. b. Read: Display records in a basic HTML table. c. Update: Concept of editing an existing record. d. Delete: Removing a record by its ID with basic confirmation. 6.4 Security Awareness for Server-Side Code i. Risks of SQL injection and the role of prepared statements in preventing unsafe queries. ii. The importance of validating and sanitizing all user input before processing. iii. A brief conceptual distinction between authentication and authorization.
--	--

4. Major Teaching and Learning Strategy

- *Lectures and Interactive Discussions:* Each topic begins with a clear explanation to help students get comfortable with the core idea, followed by space for open discussion. Students are encouraged to ask questions, share experiences, and relate what they learn to the digital tools they already use in education. The goal is to make web concepts feel approachable and connected to real life rather than purely technical.
- *Hands-On Coding Exercises:* Students spend a good portion of the course actually building things. They start by structuring pages with HTML, move on to styling with CSS, add behavior with JavaScript, and later experiment with simple PHP scripts and database interactions. Every activity is broken into small, manageable steps so students can grow their skills with confidence. For those who are curious, optional extension tasks introduce the ideas behind modern libraries and frameworks, not to master them, but to help students see where today's web development is heading.

- *Project-Based Learning*: Students should work in small groups to create a simple web application that brings together the concepts from the course. This could include a clean interface built with HTML/CSS, interactive elements powered by JavaScript, and a basic data feature supported by PHP and MySQL. The project invites collaboration, creativity, and problem-solving, skills that matter just as much as the technical ones.
- *Ethics and Real-World Case Studies*: Use brief, practical case studies and guided discussions on ethics, accessibility, data privacy, and security. This helps students recognize the wider impact of their work and encourages thoughtful, inclusive web design decisions.

5. Assessment Plan

In-Semester: (50 %)

Written exam, assignment, lab works, and mini project.

End-Semester: (50 %)

MCQ, written exam.

Reference Books:

- Felke-Morris, T. A. (2020). Web Development and Design Foundations with HTML5 (10th ed.). Pearson. ISBN: 9780136662402.
- Shklar, L., & Rosen, R. (2009). Web Application Architecture: Principles, Protocols and Practices (2nd ed.). Wiley. ISBN: 978-0-470-51860-1.
- McDonald, M. (2020). Web Security for Developers. No Starch Press. ISBN: 978-1593279943.
- Mathiason, J. (2008). Internet Governance: The New Frontier of Global Institutions (1st ed.). Routledge. ISBN: 9780415774031.
- Harold, E. R., & Means, W. S. (2004). XML in a Nutshell (3rd ed.). O'Reilly Media, Inc. ISBN: 9780596007645.
- Nixon, R. (2021). Learning PHP, MySQL & JavaScript (6th ed.). O'Reilly Media, Inc. ISBN: 9781492093824.

Official Documentation Links:

- <https://developer.mozilla.org/en-US/docs/Web/HTML>
- <https://developer.mozilla.org/en-US/docs/Web/CSS>
- <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- https://developer.mozilla.org/en-US/docs/Web/XML/XML_introduction
- <https://www.w3.org/XML/>
- <https://www.php.net/manual/en/>
- <https://www.json.org/json-en.html>
- <https://api.jquery.com/>
- <https://react.dev/>
- <https://angular.dev/>